

Dokumentation (Helen Leubner)

Matrikelnummer 9066377

In meinem Teil der Arbeit, ging es hauptsächlich um die grafische Realisierung der Startseite, sowie des Leaderboards. Außerdem habe ich mich mit der Umsetzung des User Managements und der Logik hinter dem Scoreboard, sowie den Punkten auf dem Spielfeld auseinandergesetzt. Am Ende habe ich mich noch um die Umsetzung der Musik gekümmert. Im folgenden werde ich meine einzelnen Klassen und Funktionen näher beschreiben und meine gewählten Lösungsansätze erläutern. Dabei werde ich auf die wichtigsten Funktionen im Detail eingehen.

Part 1 -> Beschreibung der Software

Meine Hauptteil liegt in dem Ordner **Display**. Dieser ist für die grafische Darstellung und die Benutzerinteraktion im Spiel verantwortlich. Er enthält alle Komponenten, die das Aussehen des Spiels und die Anzeige von Informationen steuern. Dazu gehören:

- **Startbildschirm:** Das Startmenü, in dem der Spieler das Spiel starten, Anleitungen ansehen oder das Scoreboard aufrufen kann.
- **Scoreboard:** Die Highscore-Liste, in der die Punktestände der Spieler dauerhaft gespeichert und angezeigt werden.
- **UserManager:** Verwaltung der Benutzereingabe, insbesondere die Namenseingabe zu Beginn des Spiels und das Speichern/Laden der Benutzerdaten.
- **Instructions:** Anzeige der Spielanleitung mit Hinweisen zur Steuerung und zu den Spielregeln.
- **Points** Anzeige der Points auf dem Spielfeld und Entwicklung der Collision Detection.
- **Musik** Abspielen von Musik während des Spiels, sowie ein Special Sound im Game_over case.

Implementation Startbildschirm

Die Klasse **StartScreen** ist für die Darstellung und Steuerung des Startbildschirms im Pacman-Spiel verantwortlich. Sie zeigt das Hauptmenü an, in dem der Spieler zwischen verschiedenen Optionen wählen kann. "START GAME", "CONTROLS", "HIGH SCORE" und "EXIT".

Wichtige Funktionen und Merkmale:

Zentrale Methoden:

- **draw():** Zeichnet den gesamten Startbildschirm inklusive Titel, Figuren und Menü.
- **update():** Verarbeitet die Benutzereingaben zur Navigation im Menü.
- **isOptionSelected():** Gibt zurück, ob der Spieler eine Option ausgewählt hat.
- **getSelection():** Liefert den Index der aktuell ausgewählten Menüoption.
- **Optionenauswahl:**
Mit ENTER kann der Spieler die aktuell ausgewählte Option auswählen. Die Methode **isOptionSelected()** prüft, ob eine Auswahl getroffen wurde. Dabei prüft er ob die Enter_Taste betätigt wurde. Im Anschluss wird die Funktion **getSelection()** aufgerufen. Diese gibt den Index der gewählten Option zurück. Über diesen Index wird in der Klasse **game.cpp** mithilfe eines switch-case

Block die ausgewählte Anwendung gestartet. Ich habe mich für einen Switch-Case und gegen eine if-Abfrage entschieden, weil dieser viel übersichtlicher als eine Verschachtelung von if-else-Schleifen ist. Außerdem kann man so einfacher neue Fälle hinzufügen. Auch im punkto Performance ist ein swicht-case besser, da er intern mit einer sogenannten Sprungtabelle arbeitet. Dadurch wird es ermöglicht, dass man nicht jede Bedingung einzeln prüfen muss. Vielmehr kann man direkt an die richtige Stelle im Codeblock springen und diese ausführen.

Implementation Score Board

Die Klasse **Scoreboard** ist für die Verwaltung und Anzeige der Highscores im Spiel zuständig. Sie speichert die Punktestände der einzelnen Spieler dauerhaft in einer externen Datei und lädt diese beim Programmstart wieder ein. Jeder Spielernamen wird einem Punktestand zugeordnet, sodass die Ergebnisse verschiedener Nutzer erhalten bleiben.

Zentrale Methoden

- **LoadScores()**
- **SaveScores()**
- **AddUser()**
- **IncreaseScore()**
- **GetScore()**
- **drawScore()**

Beschreibung der zentralen Funktionen

1. In der Funktion **LoadScore()** wird als Erstes überprüft, ob das Dokument scores.txt gefunden werden kann. Dieses liegt im Build-Ordner wodurch man keinen Dateipfad angeben muss. Der Name der Datei ist ausreichend. Wenn er die Datei nicht findet, gibt die Funktion eine TraceLog-Fehlermeldung zurück und springt zurück an die vorherige Stelle im Code, wo die Funktion aufgerufen wurde. Durch die Fehlermeldung kann man im Anschluss nachvollziehen, warum der Code fehlerhaft oder gar nicht ausgeführt wurde.
2. Die Funktion **SavesScores()** ist für die Speicherung des Punktestandes zuständig. Als Erstes wird die Datei geöffnet. Mit dem Befehl "out" kann man nun in die Datei schreiben. Das passiert in der for-Schleife. Diese läuft so lange, bis die Map, in welcher die Scores mit Name und int-Wert gespeichert sind, fertig ist. So lange schreibt sie in die geöffnete Datei den Namen + Score und macht am Ende eines jeden Wertepaars einen Zeilenumbruch.
3. In der Funktion **AddUser()** wird als Allererstes der aktuelle User auf den übergebenen Username gesetzt. Diesen gibt man ein wenn man ein Spiel startet. Im Anschluss wird geschaut ob es den User schon gibt. Wenn das nicht der Fall ist wird er mit dem Score von 0 Punkten in die Map hinzugefügt. Ich benutze hier im Hintergrund eine Map, weil es für mich so einfacher ist Wertepaare gemeinsam zu speichern. Eine Map arbeitet intern mit einem Schlüssel-Wert-System. Das bedeutet, dass in unserem Fall der Schlüssel der User ist und der Score der Wert, welcher dem Benutzer zugewiesen wird. Eine Map hat gegenüber eines Arrays weitreichendere Funktionen, die zum Beispiel auch die **find()** Methode umfassen welche auch in dieser Funktion genutzt wurde.
4. Als Letztes will ich hier noch auf die Funktion **drawScore()** eingehen. Als Erstes wird hier die Funktion **ClearBackground(Black)** aufgerufen. Das ist notwendig, weil es ansonsten zu Überschneidungen mit anderen Klassen geben könnte. Im nächsten Schritt wird die Map mit meinen Namen und Scores in einen Vektor umgewandelt. Dieser Schritt ist notwendig, um die Liste absteigend nach Scores sortieren zu können. Dadurch wird der User mit der höchsten Punktzahl als Erstes angezeigt. Die Ausgabe der

sortierten Punkteliste geschieht über eine for-Schleife. Dabei wird mit einer switch-case Abfrage den ersten 3 Rängen eine gesonderte Farbe zugewiesen. Der erste Platz Gold, zweiter Silber und dritter Bronze. Die restlichen Scores werden standardmäßig in Gelb angezeigt. Das entspricht dem Farbkonzept unseres Spiels. Die einzige Ausnahme bildet hier noch der currentUser. Dieser wird für eine übersichtlichere Anzeige in Grün dargestellt. **Zusammenfassung:** Die Scoreboard-Klasse sorgt dafür, dass die Highscores der Spieler dauerhaft gespeichert, verwaltet und übersichtlich angezeigt werden. Sie ermöglicht es, die Leistungen verschiedener Nutzer miteinander zu vergleichen und motiviert so zu neuen Bestleistungen.

Implementation UserManager

Die Klasse **UserManager** ist für die Verwaltung der Benutzereingabe und der Nutzerdaten im Spiel zuständig. Sie ermöglicht es dem Spieler, zu Beginn des Spiels einen Namen einzugeben, und sorgt dafür, dass dieser Name während des Spiels verwendet und zusammen mit dem Punktestand gespeichert wird.

Wichtige Funktionen und Merkmale:

- **inputName()**
- **isNameEntered()**
- **drawNameInput()**
- **getCurrentUser()**
- **reset()**
- **SaveScore()**

1. Die Funktion **inputName()** überprüft als Erstes ob bereits ein Name eingegeben wurde. Nur wenn das nicht der Fall ist, darf ein Name eingegeben werden. Für die Eingabe des Namens wird eine Integer Variabel (key) angelegt. Diese speichert den ASCII-Wert, welchen die Methode GetCharPressed() zurückgibt. In der while-Schleife werden alle gedrückten Zeichen registriert. Eine if-Abfrage geht dabei sicher, dass der Platz für die Eingabe noch ausreicht. Wenn die Backspace-Taste gedrückt wird, wird das letzte Zeichen in nameBuffer entfernt und durch ein neues "\0" ersetzt. Mit Enter bestätigt man seine Eingabe. Der Inhalt von nameBuffer wird hierbei in den String currentUser übertragen und nameEntered wird auf true gesetzt.
2. Die Klasse **saveScore()** ist hier nochmals aufgelistet. Da sie in dem Kontext des UserManagers andere Funktionen benötigt als in dem einfacheren Aufruf im Scoreboard. Hier wird sich als Erstes der bestehende Score geholt. Dabei liest die Funktion wieder zeilenweise aus der Textdatei "scores.txt" die Daten ein. Der jeweils beste Score eines jeden Users wird in der Map bestScores gespeichert. Wenn jetzt der Score der gespeichert wird besser ist, als der bisherige Score des aktuellen Users wird dieser als neuer Score des currentUser gespeichert. Anschließend wird die Datei im Schreibmodus geöffnet und im ersten Schritt einmal komplett geleert. Danach werden alle Einträge der Map erneut in die "scores.txt" Datei geschrieben.
3. Die Klasse UserManager enthält auch eine **reset()** Funktion. In dieser wird der aktuelle User zurückgesetzt und der boolsche Variabel, welche anzeigt ob ein Benutzername eingegeben ist wird auf false gesetzt. Das ermöglicht es einen neuen Nutzer anzulegen. **Zusammenfassung:** Die UserManager-Klasse sorgt dafür, dass jeder Spieler eindeutig identifiziert werden kann und sein Punktestand dauerhaft gespeichert wird. Sie bildet die Schnittstelle zwischen Benutzereingabe und Highscore-Verwaltung und ermöglicht so eine individuelle Zuordnung der Ergebnisse zu den jeweiligen Spielern.

Implementation Instructions

Die Klasse **Instructions** ist für die Anzeige der Spielanleitung im Pacman-Spiel zuständig. Sie stellt dem Spieler übersichtlich alle wichtigen Informationen zu Spielregeln, Steuerung und Zielen zur Verfügung. Die Anleitung wird grafisch mit Raylib dargestellt und enthält Hinweise zu Punkten, Levelaufstieg, Geistern, Steuerung und Highscore. Über die Methode **draw()** wird der Anleitungstext auf dem Bildschirm angezeigt. Mit der Methode **update()** kann der Spieler per Tastendruck (ENTER) zur vorherigen Ansicht zurückkehren. So sorgt die Klasse dafür, dass neue Spieler schnell einen Überblick über das Spiel erhalten und jederzeit auf die wichtigsten Regeln zugreifen können.

Implementation Points

Die Klasse Points verwaltet alle Sammelpunkte (kleine gelbe Punkte) auf dem Spielfeld, die Pacman im Laufe des Spiels einsammeln muss. Sie sorgt dafür, dass die Punkte korrekt platziert, angezeigt und entfernt werden, sobald Pacman sie einsammelt. **Wichtige Funktionen und Merkmale:**

- **Point()**
- **draw()**
- **hasRemainingPoints()**
- **isPointAllowed()**
- **setPoint()/getPoint()**
- **wrapX()/wrapY()**

1. Dem Konstruktor **Point()** wird als Erstes das Objekt checkPos vom Typ CheckPosition übergeben. Diese beinhaltet die Funktion **isWalkableForPacman()**. Außerdem werden Dummy-Werte initialisiert. Da die Klasse Points nicht ein einzelnes Item darstellt sondern ein ganzes Raster von Punkten sind die Positionswerte hier erstmal nicht relevant. Als Nächstes wird die Levelgröße abgefragt und die Breite, sowie Höhe in Integer Variablen gespeichert. Im Anschluss werden alle Zellen erstmal mit **false** initialisiert, was bedeutet, dass sich hier erstmal keine Punkte befinden. Erst danach wird das Feld durch eine zweifach verschachtelte for-Schleife, sowie eine if-Abfrage mit Punkten befüllt. Dabei wird mit der äußeren for-Schleife die y-Richtung und mit der inneren Schleife die x-Achse abgelaufen. An jedem Punkt(x/y) wird geprüft, ob dieser Punkt begehbar für Pacman ist. Außerdem mussten wir die Überprüfung einführen, ob es erlaubt ist, dort einen Punkt hinzusetzen. Das liegt am Spieldesign. Der Startpunkt der Geister sowie die Portale sind beide begehbar für Pacman. Jedoch sollen da keine Punkte liegen. Deswegen war es notwendig die zusätzliche Funktion **isPointAllowed** hinzuzufügen.
2. Ebenfalls wichtig ist die Funktion **hasRemainingPoints()**. In dieser wird geprüft ob auf dem Feld noch ein Punkt liegt. Das verhindert, dass Punkte zweimal eingesammelt werden können.

Zusammenfassung Die Klasse Points ist zentral für die Spielmechanik von Pacman: Sie verwaltet, wo Punkte liegen, ob sie schon eingesammelt wurden, und sorgt für das Zeichnen und Zurücksetzen der Punkte. Damit ist sie ein wichtiger Bestandteil für das Levelziel und die Punktevergabe im Spiel.

Implementation der Musik

Diese Funktion hat keine eigene Klasse. Sie ist vielmehr, in ihrer Funktion über die Game-Klassen verteilt. Für die Umsetzung habe ich mich nach einigem Ausprobieren für die Ausführung mit Raylib entschieden. Raylib verwendet intern die Bibliotheken miniaudio, sowie dr_wav. Damit wird ein kleiner Umfang von Funktionen bereitgestellt um kürzere bis mittellange Sounds abzuspielen. Hierbei ist es wichtig, dass man die Funktion **InitAudioDevice()** aufruft. Über diesen Aufruf wird das Backend miniaudio gestartet und die Standard-

Audiogeräte, wie zum Beispiel ein Lautsprecher initialisiert. Im Anschluss muss die Musik aus einer wav-Datei geladen werden und die Funktion **PlayMusicStream()** aufgerufen werden. Um die Musik während der gesamten Spieldauer abspielen zu lassen muss man solange man in dem case Playing ist die Musik mit einem **UpdateMusicStream()** aktualisieren. Am Ende ist es wichtig den Musik-Stream und das AudioDevice zu schließen. Ansonsten kann es zu Überlagerungen und fehlerhaften Aufrufen kommen.

Part 2 -> KI Hinweis

Werkzeug	Beschreibung der Nutzung
Chat GPT	1. Einrichtung von VS-Code sowie Implementierung des Compilers MINGW und der Bibliothek Raylib 2. Entwicklung von Code 3. Überprüfung und Lösen von Fehlern
Copilot	1. Überprüfung von Fehlern 2. Hilfe beim Schreiben der Dokumentation (Erklären von Funktionen sowie Überprüfung auf Fehler)

Testtabelle

Testcase	Erwartete Reaktion	Gemessene Reaktion
Funktion Menü	Der User kann alle Menüpunkte auswählen und es öffnet den Inhalt dahinter korrekt	Anfangs Schwierigkeiten beim Score. Grund dafür war eine doppelte Datei. In die eine wurde geschrieben und aus der anderen gelesen. So kam es Zustände das nie ein Score angezeigt wurde
Korrektes abspielen der Musik	Die Musik soll sobald der Benutzer seinen Namen eingegeben hat starten. Wenn er alle 3 Leben verloren hat wird die Spielmusik gestoppt und die Game_Over Musik gestartet.	Es traten Probleme auf, dass nur für eine kurze Zeitspanne (ca 1sec) Musik abgespielt wurde. Grund falscher Ladebefehl und fehlerhafter Aufruf (nicht die richtige Stelle im Code)
Speichern der Scores	Die Scores sollen sobald der Spieler das Spiel beendet hat, in der Datei scores.txt gespeichert werden	Probleme mit direktem Abruf des Highscores nach einem Spiel. Grund fehlerhafter Funktionsaufruf der saveScore() Funktion
Selber Nutzer erzielt höheren Highscore	Wenn ein Nutzer erneut mit dem selben Namen spielt und einen höheren Score als sein bisherigen Highscore erzielt, soll dieser in der Tabelle überschrieben werden	Wie erwartet
Collision Detection Points	Immer wenn Pacman sich auf dem selben Feld, wie noch ein existierender Punkt befindet, soll sich der Score um 10 erhöhen und der Punkt verschwinden	Wie erwartet

Konzepte aus dem Kurs

Im folgenden will ich nochmal Explezit auf die genutzen Konzepte eingehen, welche wir im Kurs behandelt haben. Als erstes ist hier das Konzept der Vererbung zu nennen. Meine Klasse Point erbt von den Klassen items, config und Collision_Detection. Das ermöglicht es mir, bestimmte Funktionen und Werte, wie zum Beispiel die Größe des Spielfeldes abzurufen ohne es ein zweites mal implementieren zu müssen. Außerdem Kapsle ich Variabeln in den einzelnen Klassen. Dadurch wird die Integration des Objektes gewährt. Die Variabeln sind auf privat gesetzt und können nur über Getter/Setter aufgerufen werden. In meinem Ordner Display benutze ich jedoch keine Polymorphie. Das ist auch nicht nötig, weil alle Klassen unabhängig sind und einzeln im Code aufgerufen werden. Es gibt keine Sammlung von Screens, welche polymorph verwaltet werden müssten. Auch meine Point Klasse erbt nicht von einer Basistklasse mit einer virtuellen Methode, welche ich überschreiben müsste. Bedeutet auch hier habe ich keine Polymorphie genutzt. Der Grund ist der Aufbau unserer Projektstruktur in die ich meinen Part integriert habe.

Entwicklungsprozess

Vorbereitung

Angefangen habe ich das Projekt mit dem Download von Raylib und der Einrichtung von VS-Code. Das hat mich aufgrund von verschiedenen Problemen leider einige Zeit gekostet. VS-Code hat sehr lange die Raylib Bibliothek nicht akzeptiert. Nach verschiedenen Versuchen und mehrfachen Installieren und Deinstallieren von Raylib und dem Compiler MINGW hat es irgendwann funktioniert und ich konnte mit meinen eigentlichen Aufgaben starten. Hierfür habe ich für unser Projekt erstmal für einen groben Überblick ein Klassendiagramm erstellt. Außerdem haben Thea und ich uns einmal ganz am Anfang zusammengesetzt, um zu besprechen, wer was macht und wie wir was haben wollen.

Entwicklung

Gestartet habe ich mit der Point Klasse. Hier war es am Anfang erstmal wichtig, dass überhaupt Punkte auf dem Spielfeld erscheinen. Erst im zweiten Teil habe ich mich um eine Collision Detection und den Score gekümmert. Die Schwierigkeit lag dabei darin, dass die Punkte nicht auf alle Stellen im Spielfeld gezeichnet werden sollten. Gelöst wurde das Problem durch das hinzufügen von Funktionen, welche die unerwünschten Positionen herausfilterten. So konnte man eine Abfrage machen und nur wenn diese Position sowohl begehbar für Pacman ist, sowie keiner "verbotenen Zone" entspricht darf dort ein Punkt gezeichnet werden. Im Anschluss an die Point Klasse habe ich mich mit dem Startbildschirm beschäftigt. Da es hier nicht viel Logik zu implementieren gab, war es hauptsächlich das Design, welches hier Arbeit verursachte. Bei dem Scoreboard sah das wieder anders aus. Dort war es am Anfang schwierig die Scores korrekt abzurufen. Direkt nach einem Game sah man auf dem Scoreboard nur den jeweiligen Benutzernamen gefolgt von einem Score von 0. Wenn man das Spiel einmal schloss und dann wieder öffnete, wurden die korrekten Highscores angezeigt. Der Fehler lag in der Speicherung des Scores. Dieser wurde an der falschen Stelle aufgerufen. So passierte es, dass zur Laufzeit noch nicht der korrekte Score angezeigt wurde, sondern nur der default Wert von 0.

Ausblick in die Zukunft

Für die Zukunft würde ich für meinen Part noch eine Musik für den Übergang von einem Level in das nächste, sowie ein verlorenes Leben hinzufügen. Außerdem würde ich das Scoreboard nochmals optisch überarbeiten, sodass man noch leichter und übersichtlicher auf einen Blick die aktuellen Highscores angezeigt bekommt.